ArcWarden: a GPU-native particle-in-cell framework for high-throughput kinetic plasma simulations

Donglai Ma
Department of Atmospheric and Oceanic Sciences, UCLA
dma96@atmos.ucla.edu

Draft v0.2 — August 12, 2026

Abstract

We present ArcWarden, a GPU-native 2D3V particle-in-cell (PIC) framework designed for high-throughput kinetic plasma simulation on a single modern GPU. Rather than porting the established multi-node PIC design to accelerators, ArcWarden is built around what current GPUs provide: a last-level cache large enough to hold the entire 2D field state, fast shared-memory atomics, and inexpensive wide kernel launches. The particle step is a single fused kernel (field gather, Boris push, current scatter, and migration), currents accumulate in shared-memory tile aprons, and the particle sort that enables tiling is amortized over tens of steps and *stale-tolerant* — correctness never depends on sort recency. On the same physical problem with matched physical parameters and diagnostics (an anisotropy-driven whistler and electron-acoustic-wave problem with 156 million particles and 2.1×10^5 steps), ArcWarden is 2.6× faster than the CUDA branch of the OSIRIS framework on the same device, sustaining 1.55×10^{10} particle-steps per second (the codes differ in interpolation order — quadratic vs linear-plus-filter — a difference we bound at the 7% level by direct measurement). A modular architecture lets electrostatic, spectral-Darwin, and full-Maxwell field solvers — together with inhomogeneous background fields, loss-cone and anisotropic distributions, δf and cold-fluid species, and absorbing boundaries — share one tested 2D GPU particle engine (a specialized 1D electron-hybrid engine shares the framework's building blocks), with every physics setup expressed as a runtime input deck. ArcWarden passes a three-level validation ladder: textbook verification (dispersion, growth rates, conservation), reproduction of the published nonlinear trapping simulations of An et al. [PRL 122, 045101 (2019)], and, as advanced capability demonstrations, self-consistent generation of coherent whistler-mode waves with rising-tone frequency chirping in an inhomogeneous magnetic field — both triggered, following Tao et al. [GRL 44, 3441 (2017)], and spontaneous from loss-cone free energy in a dipole geometry, reproducing the whistler regime taxonomy of Chen et al. [Phys. Plasmas 33, 072105 (2026)].

Contents

1 Introduction

Kinetic simulation is the primary tool for collisionless plasma physics at electron scales: whistler-mode chorus and hiss, electron-cyclotron harmonic waves, electron-acoustic waves, and the associated time-domain structures all require a particle-in-cell (PIC) treatment [1, 2]. The questions are statistical — growth from shot noise, nonlinear saturation, resonant trapping — and demand hundreds of particles per cell over 10^5–10^6 time steps. Raw particle throughput (particle-steps per second) therefore decides which problems are reachable, and how many points of a parameter scan a researcher can afford.

Mature GPU-capable PIC frameworks exist — OSIRIS [3], VPIC [4], PIConGPU [5], WarpX [6], and the algorithm studies of Decyk and Singh [7]. They are engineered for a specific and important regime: very large problems decomposed across many nodes, with the GPU port grafted onto a design vocabulary formed by the hardware of the early 2010s — small caches, slow global atomics, and expensive kernel launches, answered by tiling the grid, binding particles to tiles in fixed-size chunks, and paying for that locality with per-step particle bookkeeping (find the movers, exchange, compact).

Meanwhile the hardware under a single workstation has changed character. A consumer NVIDIA Blackwell GPU (RTX 5090) offers 96 MB of L2 cache, 1.8 TB/s of memory bandwidth, 32 GB of device memory, hardware-accelerated shared-memory atomics, and kernel launches cheap enough to run several wide kernels per PIC step without measurable gaps. For 2D electron-scale problems the *entire field state* now fits in cache several times over, and 10^8–10^9 particles fit in device memory. This opens room for a different design philosophy:

> *a specialized, GPU-native kinetic simulation framework that optimizes time-to-science on one GPU — high single-device throughput, plus an architecture in which new physics capabilities are added as runtime modules rather than code forks.*

ArcWarden is that framework. It makes three claims, which organize this paper:

1. **Performance** (Section 5). On the same physical problem with matched parameters and diagnostics, ArcWarden completes a 156-million-particle, 207,000-step kinetic run 2.6× faster than OSIRIS-CUDA on the same GPU; an ablation chain attributes the gain to each design element, and direct measurements show the classic chunk-pipeline design is overhead-bound, not compute-bound, on cache-rich hardware.

2. **Extensibility** (Section 6). One tested GPU particle engine hosts three field models (electrostatic, spectral Darwin, full-Maxwell Yee), full-f and δf kinetic species, a linearized cold-fluid species, prescribed inhomogeneous background fields, loss-cone and anisotropic loading, and absorbing particle and field boundaries — all selected at runtime by plain-text input decks.

3. **Physics capability** (Section 7). The framework passes a three-level validation ladder: textbook verification; reproduction of the published nonlinear whistler-pump trapping simulations of An et al. [8] from their table of parameters; and self-consistent generation of coherent whistler waves with rising-tone frequency chirping in an inhomogeneous magnetic field — triggered, following the setup of Tao et al. [9], and spontaneous, reproducing the whistler regime taxonomy of Chen et al. [10] in a compressed dipole field.

ArcWarden does not aim to replace massively parallel general-purpose PIC frameworks; Section 8 states the division of labor explicitly. The niche it targets — electron-scale kinetic studies that fit one modern GPU, run in minutes to hours, and change physics configuration daily — is, we argue, both large and underserved.

2 ArcWarden architecture

2.1 Overall design

ArcWarden is a C++20/CUDA code for a single GPU, organized as a header-only library templated on a compile-time configuration (spatial and velocity dimensionality, particle shape, precision, deposit policy) with thin translation units for kernels. A simulation is defined entirely by a runtime *input deck* (INI text): grid, time step, field model, species list, background field, drivers, boundaries, and diagnostics. Adding an experiment is a new deck, never a new `main()`. The layering is:

input deck (`*.ini`)
$\downarrow$
physics configuration (field model, species, B_0, boundaries, drivers)
$\downarrow$
core PIC engine (particles · field solver · diagnostics)
$\downarrow$
GPU kernels (fused push–deposit, sort, spectral/FDTD solves)

[replace with a drawn architecture figure (Fig. 1): deck → configuration → core → kernels, with the physics-module column of Table 5 hanging off the configuration layer]

Fields and moments live on a 2D grid; particle momenta keep all three components (2D3V). All physics uses $\omega_{pe} = 1$, $m_e = 1$, $\varepsilon_0 = 1$ normalization (the 1D chirping module uses $\Omega_{e0} = c = 1$, the convention of the chorus literature).

2.2 Particle representation and memory layout

Particles are a structure of arrays — positions, momenta $\mathbf{u} = \gamma\mathbf{v}/c$, cell index, and (in the δf branch) marker weights — in 32 bit floating point, with all reductions and diagnostics accumulated in 64 bit. Every owning container exposes a POD *view* (`DeviceView`: pointer + size) that is passed to kernels by value, keeping kernels free of host types and ownership. Loading supports quiet starts (coherent sub-cell velocity bases, flat initial charge density) and noisy starts from a counter-based RNG, per species.

The central memory-layout decision is that **fields live in L2 cache**. The full 2D field state — six Yee components plus three current components at 625^2 in single precision — is ≈ 14 MB, 15% of the 96 MB L2 on the benchmark device. Random-order particle gathers are L2-served after first touch, so ArcWarden performs *no* shared-memory staging of field tiles — the core trick of classic GPU-PIC. A microbenchmark shows staging is in fact a net loss on this hardware: a shared-memory-staged push runs at 0.82× the plain L2-served push.

2.3 Fused gather–push–deposit kernel

The entire particle update is one kernel: (i) staggered linear interpolation of E and B from the grid (global loads, L2-served) plus the analytic background B_0; (ii) Boris rotation and position update; (iii) Esirkepov current scatter over the 4×4 union stencil; (iv) migration fused into write-back — periodic wrap and cell-index recompute as the position is stored, exactly once. A full PIC step is 4–5 wide kernels with no host round-trips; at $\sim$3 µs launch cost against $\sim$10 ms of work per step, launch gaps are unmeasurable, so no megakernel or persistent kernel is needed.

2.4 Current deposition: shared-memory tile aprons

The scatter is the classic PIC bottleneck: with global atomics each CIC/Esirkepov particle issues $\sim$20 atomic adds to scattered addresses. ArcWarden tiles the grid into 16×16-cell tiles; each tile's thread block accumulates current in a shared-memory apron (tile + drift margin $D = 2$ + stencil halo), then flushes it to global memory once per block ($\sim$2k cells) — a cost independent of the number of particles in the tile. The scatter core is one templated routine, `esirkepov_scatter<Sink>`; the `Sink` policy selects shared-tile or global-atomic accumulation, so the fast path and the fallback share a single tested implementation:

Listing 1: Structure of the fused particle kernel (one block per 16×16-cell tile).

```
__global__ k_push_esirkepov_tiled(particles, bins, fields, ...):
  __shared__ float s_jx[..], s_jy[..], s_jz[..]; // tile + apron
  zero shared tile; __syncthreads();
  for each particle t of this tile (grid-stride over the bin):
    (E,B) = staggered_gather(fields, x0, y0); // global loads -> L2
    u = boris(u, E, B + B0(x0,y0)); // push + background
    (x1,y1) = (x0,y0) + v*dt;
    if stencil(x0..x1) inside apron:
        esirkepov_scatter<SharedJSink>(...); // fast path
    else:
        esirkepov_scatter<GlobalJSink>(...); // stale-sort stray
    store wrapped positions + recomputed cell index; // fused migration
  __syncthreads();
  flush shared apron to global J (one atomicAdd per apron cell);
```

2.5 Particle sorting: amortized and stale-tolerant

Tile locality comes from a physical counting sort (a reorder of all particle arrays keyed on cell index) that runs every $N_{\mathrm{sort}} \sim 20$ steps — not per step. Two facts make this safe and fast:

- **Correctness does not depend on sort recency.** A particle that has drifted beyond its tile apron since the last sort is detected at scatter time and deposits through the global-atomic sink. The charge-conservation test gates exactly this path: with a deliberately 4-step-stale sort and active strays, the discrete continuity residual is 4.6×10^{-6}.
- **The sort amortizes to noise.** The full sort costs 12.4 ms; at $N_{\mathrm{sort}} = 20$ that is 0.6 ms per ∼10 ms step, and the end-to-end rate is flat for $N_{\mathrm{sort}} = 10$–40 (1.62–1.74×10^{10} particle-steps/s, full-length runs) — there is no cliff to tune against.

This replaces the entire per-step find-movers/exchange/compact pipeline of chunk-based designs with one amortized kernel plus a branch in the scatter; the sort cadence is purely a performance dial.

2.6 Field solvers

Three field branches share the particle engine, selected at runtime by the deck (`[field] model = es | darwin | yee`): a spectral electrostatic solver, a spectral Darwin (magnetoinductive) solver, and a full-Maxwell FDTD (Yee) solver with exactly charge-conserving Esirkepov deposition. Their numerics are given in Section 3; their roles are complementary — the Darwin branch removes the light-wave CFL limit and is the fastest route to low-frequency electromagnetic physics, while the Yee branch provides full-Maxwell ground truth. The two are cross-validated against each other (Section 4.1). All spectral solves are cuFFT-diagonal; the entire field solve at 625^2 costs < 0.2 ms per step, negligible against the particle step.

2.7 Diagnostics and I/O

Diagnostics are deck-selected modules sharing the engine's data views: mode power and $\delta B/B_0$ spectra, k–t and ω–k reductions, phase-space frames and movies, field snapshots, and energy/charge histories (accumulated in double precision). Heavy reductions run on-device; only reduced arrays cross to the host. A versioned checkpoint *format* (header, verbatim deck text, git hash, and an array manifest) is defined and format-tested; full-state streaming restart is a planned milestone and is stated as such.

2.8 Correctness gates

Every performance path in the engine is admitted only behind a `ctest` gate: *J-identity* (one tiled step reproduces the flat kernel's current field to 2×10^{-7}, worst cell); *continuity* ($\partial_t \rho + \nabla \cdot J = 0$ discretely to 4.6×10^{-6} with a stale sort and active strays); *energy parity* (tiled-path field-energy history matches the flat path to 4×10^{-7} through linear growth); and *physics reproduction* (the benchmark figures of Sections 5–7 act as end-to-end acceptance tests). The test suite runs 30+ gates from FFT round-trip up to published-figure reproduction.

3 Numerical models

3.1 Particle equations and interpolation

Particles obey the relativistic Lorentz equation, integrated with the Boris rotation [11] in leapfrog arrangement; velocities are rolled back half a step at initialization. Interpolation is linear (CIC) on gather and scatter in all results shown; the deposit framework is templated over the assignment function so that quadratic (TSC) shapes can be added with a one-cell-wider apron, but TSC is not yet implemented — all results in this paper are CIC.

3.2 Electrostatic and Darwin spectral solvers

On the periodic domain the electrostatic branch solves $\nabla^2 \phi = -\rho/\varepsilon_0$, $E_L = -\nabla\phi$ in k-space. The Darwin branch retains the full magnetoinductive field: with $E = E_L + E_T$, $\nabla \times E_L = 0$, $\nabla \cdot E_T = 0$, the Darwin approximation drops the transverse displacement current, giving the elliptic system

$$\nabla^2 \phi = -\rho/\varepsilon_0, \qquad E_L = -\nabla\phi, \tag{1}$$

$$\nabla^2 B = -\mu_0 \nabla \times J, \tag{2}$$

$$\nabla^2 E_T = \mu_0 \, \partial_t J_T, \tag{3}$$

with (3) closed by the acceleration ($\dot{J}$) and momentum-flux ($\langle \mathbf{uu} \rangle$) moments in the standard Darwin formulation [12], solved in one shot in k-space via the resummed transverse Green's function. The electrostatic and Darwin particle path is currently non-relativistic ($\gamma \equiv 1$; appropriate for the $v/c \lesssim 0.1$ regimes shown here); the Yee and hybrid branches use the relativistic push. No light wave exists in the model, so the time step is set by $\omega_{pe}\Delta t \lesssim 0.2$ and particle transit rather than the light CFL condition — a 6.9× larger step than the Yee branch on the whistler problems of Section 7. This model lineage (UPIC) is the one used by An et al. [8], which makes their published simulations a like-for-like validation target.

3.3 Full-Maxwell FDTD solver

The Yee branch [13] advances all six field components with the full Maxwell equations; current is deposited with the Esirkepov density-decomposition scheme [14], which satisfies the discrete continuity equation to round-off, so no Poisson clean is ever required ($\nabla \cdot B$ stays at round-off; the Gauss residual stays frozen at its initial value, $\sim 2 \times 10^{-7}$). An optional binomial current filter ($[1, 2, 1]^{\otimes 2}/16$ per pass, matching the OSIRIS `smooth` block) suppresses grid-scale noise; being linear and shift-invariant it preserves the discrete Gauss law. A uniform or prescribed background field B_0 enters at the gather, so the Yee update carries only self-consistent fields.

3.4 One-dimensional electron-hybrid model

For field-aligned whistler studies the framework provides a 1D electron-hybrid branch — a specialized engine sharing the framework's building blocks (deposit stencils, δf machinery, boundary masks) rather than the 2D particle engine itself — in the Katoh–Omura model class [15, 2]: transverse fields on a staggered 1D grid, a linearized cold electron fluid carrying the whistler dispersion, relativistic kinetic hot electrons (full-f or nonlinear δf), the adiabatic mirror force of a prescribed inhomogeneous field $B_0(h) = B_0(1 + ah^2)$ (or a dipole profile), particle reflection at the domain ends, Umeda-style masked absorbing layers for the fields [16], and an equatorial triggering antenna. This is the branch used for the chirping demonstration (Section 7.2).

3.5 δf representation

Hot species may be represented as δf markers about an analytic equilibrium f_0 (bi-Maxwellian, optionally loss-cone-subtracted, with an (E, μ)-mapped mirror equilibrium along inhomogeneous B_0); marker weights evolve along characteristics and deposit $\delta\rho, \delta J$. The δf branch is gated by equilibrium-hold tests (weights stay at machine noise in a held equilibrium) and by growth-rate agreement with full-f on the same deck.

4 Verification

The first level of the validation ladder asks only: *is the code correct?* The `ctest` suite covers, per branch:

- *Electrostatic:* cold Langmuir oscillation frequency; two-stream instability growth rate against the fluid dispersion; bump-on-tail relaxation; Landau damping (δf).
- *Darwin:* magnetostatic B-from-J against the analytic current sheet; Weibel instability growth rate.
- *Yee:* vacuum wave propagation; Langmuir oscillation on the FDTD path; cold-plasma whistler eigenmode frequency (0.03% of the full-Maxwell root); magnetized single-particle gyromotion and mirror bounce; long-run energy conservation; discrete continuity and Gauss residuals (Section 2.8).
- *Hybrid/chirping:* cold whistler dispersion on the hybrid grid; anisotropy-driven growth rate against the kinetic dispersion solver; δf equilibrium hold under inhomogeneous B_0.

Table 1 collects the quantitative gates with their measured values and references; each row is a `ctest` that must pass on every commit. [add grid-resolution and time-step convergence rows and a seed-sensitivity study — planned before submission]

Figure 1 condenses the electrostatic-branch level of the ladder into one deck-driven figure (produced by `scripts/repro/plot_verification.py`): the two-stream instability grows at $\gamma = 0.343\,\omega_{pe}$ against the cold-beam rate $\omega_{pe}/\sqrt{8} = 0.354$ (the 3% deficit is the finite $v_{th}/v_0 = 0.1$ thermal correction), saturates into the classic trapping vortex, and over a 10× longer run the total energy oscillates within $\pm 2 \times 10^{-3}$ with no secular drift while the net charge is conserved to round-off. [extend to a companion EM panel set (whistler eigenmode + Weibel growth) from the same script]

Table 1: Quantitative verification gates (RTX 5090, values as asserted by the test suite; deck/grid parameters in the named tests).

quantity	branch	measured	reference	rel. err
Langmuir frequency	ES spectral	$0.9996\,\omega_{pe}$	1.0	0.04%
two-stream growth rate	ES spectral	$0.343\,\omega_{pe}$	$\omega_{pe}/\sqrt{8}$ (cold)	3% (thermal)
whistler eigenmode	Darwin	$0.44980\,\omega_{pe}$	0.45000 (Darwin disp.)	0.05%
whistler eigenmode	Yee	$0.44885\,\omega_{pe}$	0.44897 (full Maxwell)	0.03%
inter-branch $\Delta\omega$	Darwin vs Yee	0.21%	0.23% (displacement current)	—
loss-cone linear growth	hybrid δf	$3.86 \times 10^{-3}\,\Omega_e$	3.9×10^{-3} (disp. solver)	2%
dipole bounce period	particle	—	independent RK4	0.16%
continuity residual	Yee tiled + stale sort	4.6×10^{-6}	0	—
Gauss-law drift	Yee (Esirkepov)	2×10^{-7}	0	—
J-identity (tiled vs flat)	Yee	2×10^{-7}	0	—

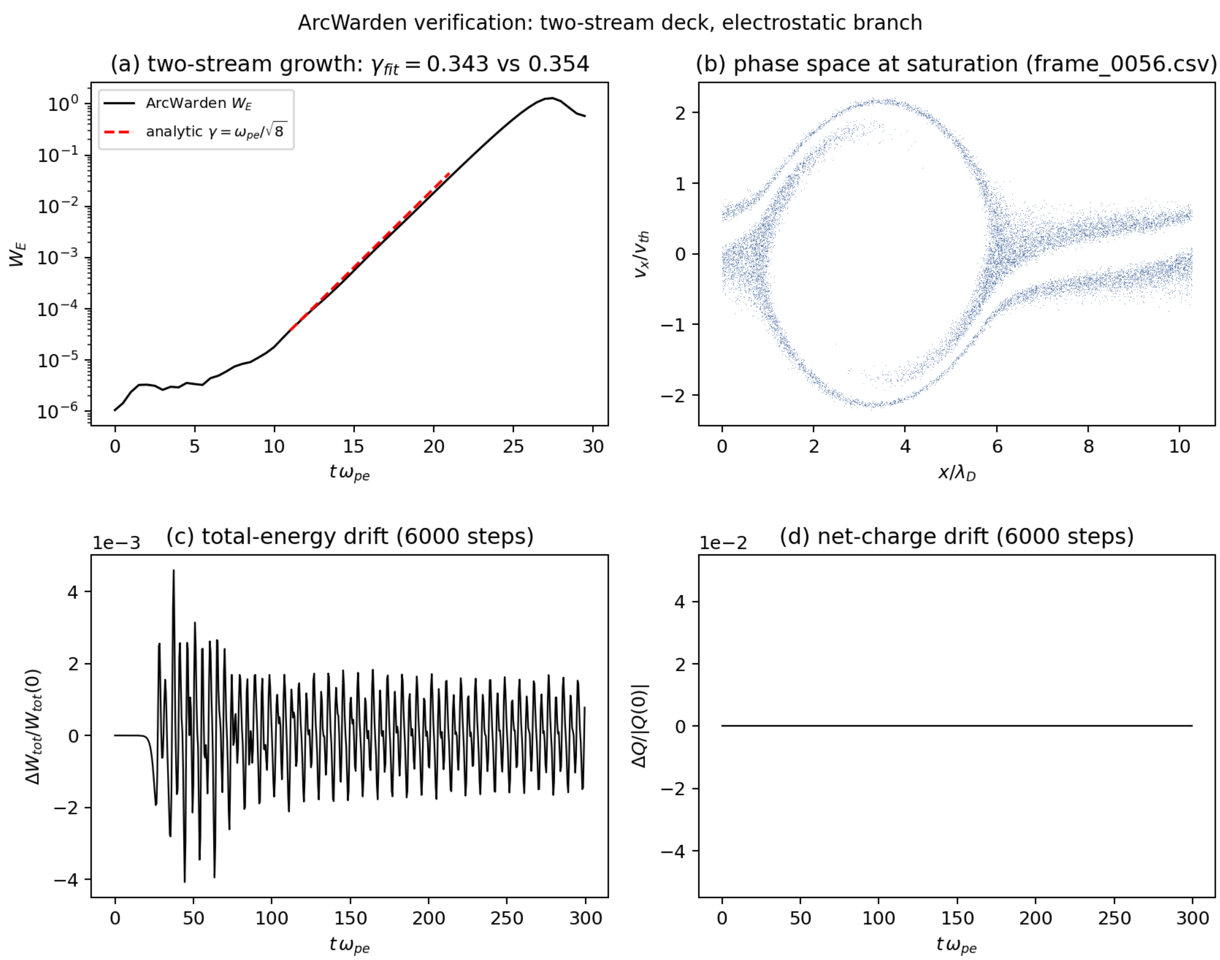

Figure 1: Electrostatic-branch verification from one input deck (`decks/verification/two_stream.ini`): (a) field-energy growth vs the analytic two-stream rate; (b) phase space at saturation; (c) relative total-energy drift and (d) relative net-charge drift over a 6000-step run.

4.1 Cross-branch validation

The Darwin and Yee branches are validated against each other where the Darwin approximation holds ($\omega^2 \ll k^2c^2$); agreement between a radiation-free spectral solver and a full-Maxwell FDTD solver sharing only the particle engine is a sensitive check on both. At eigenmode level, a cold parallel whistler at $kc = 3\,\omega_{pe}$, $\omega_{ce} = 0.5\,\omega_{pe}$ gives $\omega = 0.44980$ (Darwin; analytic 0.45000) and 0.44885 (Yee; full-Maxwell root 0.44897) — and the 0.21% gap *between* the measured frequencies matches the 0.23% analytic displacement-current gap between the two field models: the branches differ by exactly the physics that separates the models. At nonlinear level, the An et al. Sim 3 deck was run through both branches (the Yee branch at its light-CFL step, 6.25× more steps): whistler amplitude peaks at $\delta B/B_0 = 0.105$ (Darwin) vs 0.108 (Yee) with identical ring-down, and the nonlinear electrostatic band and trapping structures agree (Fig. 2).

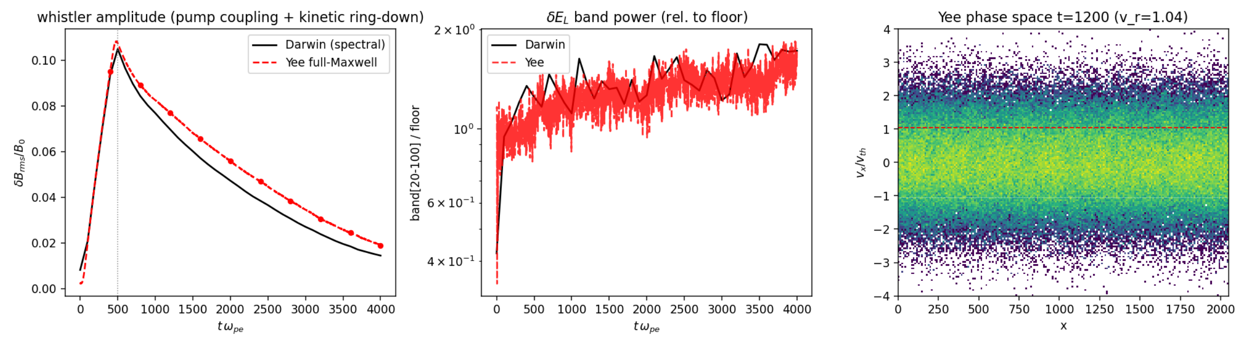

Figure 2: Darwin (spectral) vs Yee (full-Maxwell) branch on the same input deck (An et al. Sim 3): whistler amplitude history (left), secondary electrostatic band power over the noise floor (middle), and Yee-branch phase space at $t = 1200\,\omega_{pe}^{-1}$ (right).

5 GPU performance

All measurements are on one NVIDIA RTX 5090 (Blackwell, `sm_120`: 170 SMs, 96 MB L2, 31.4 GB GDDR7), CUDA 13.3. The benchmark problem is case 7 of the whistler/electron-acoustic parameter study of Ma et al. [17]: a bi-Maxwellian electron plasma, $T_\perp/T_\parallel = 5$, $\beta_\parallel = 0.0091$, $\omega_{ce}/\omega_{pe} = 0.25$, 625^2 cells, 400 particles per cell (156.25M particles), run to $t = 3000\,\omega_{pe}^{-1}$ (206,897 Yee steps).

5.1 Head-to-head against OSIRIS-CUDA

Both ArcWarden (Yee branch, linear CIC + 3-pass binomial filter) and OSIRIS 4.4.4 CUDA (quadratic + binomial-3, its production configuration) ran the case end-to-end with matched diagnostics and were reduced by the same eight-panel analysis (Fig. 3). The physics agrees panel by panel: whistler onset at $k_x \simeq 2.8\,\omega_{pe}/c$ in both, wave-normal angle 44–47° in both, the electron-acoustic band above noise by the same factor in both, the beam-mode ridge on $\omega = 0.0546c\,k_x$ in both; field-energy histories agree to four significant figures through the linear phase.

The table pairs measurements taken in the same session on the same driver. The current build remeasures the ArcWarden run at 1900.8 s (1.70×10^{10}, Table 3), which would raise the ratio to 2.8×; we quote the conservative matched-session figure and will refresh both sides together before submission. [rerun OSIRIS-CUDA on the current driver for an updated matched pair]

Table 2: End-to-end wall time, same physical problem, matched physical parameters and diagnostics (interpolation orders differ; see text). 156.25M particles, 206,897 steps, same RTX 5090.

code	wall time	particle-steps/s	speedup
OSIRIS 4.4.4 CUDA (quadratic, binomial-3)	5392.8 s	6.0×10^9	1
ArcWarden Yee tiled (linear, binomial-3)	2079.7 s	1.55×10^{10}	2.59×

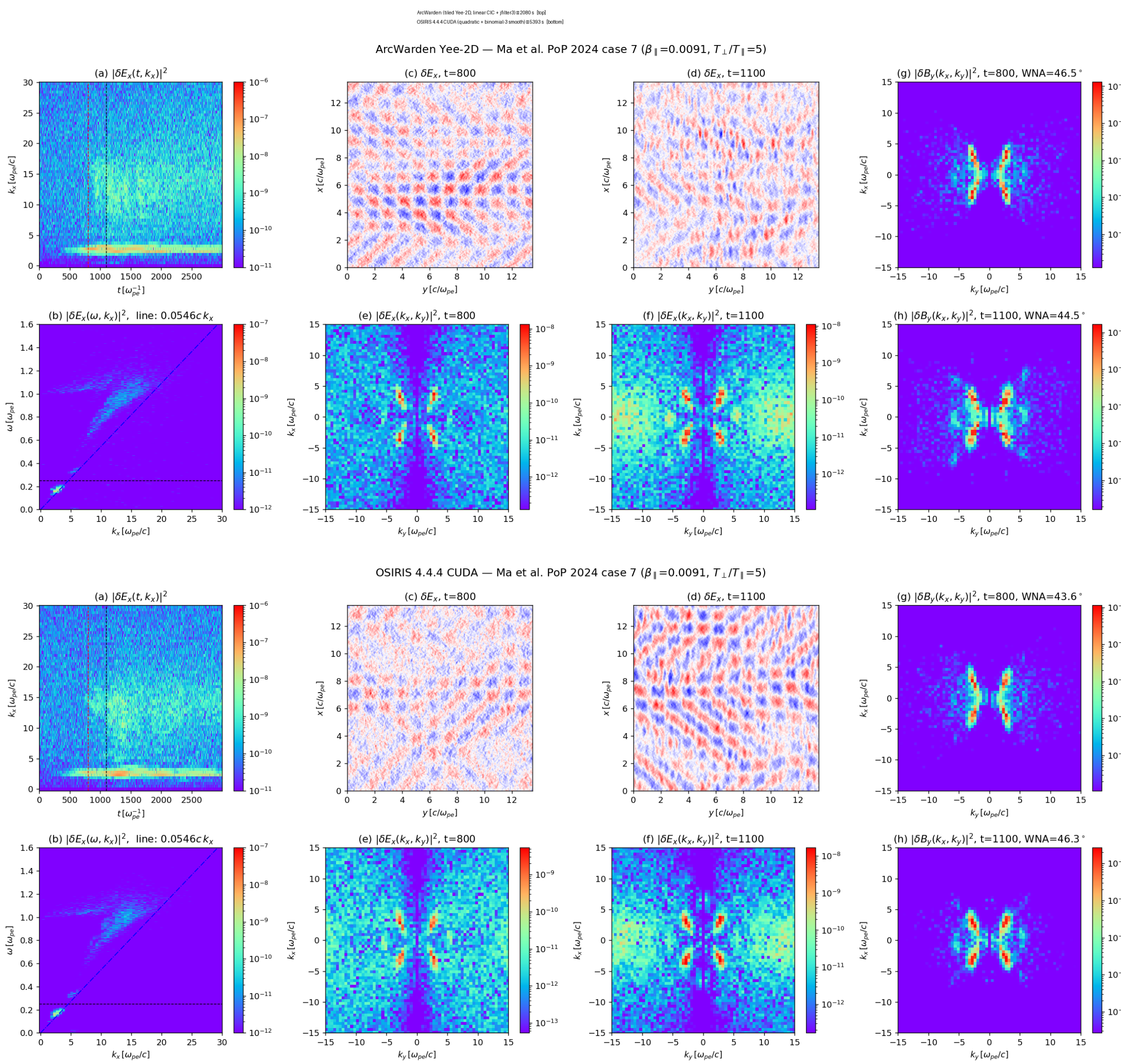

Figure 3: The benchmark case run in ArcWarden and OSIRIS-CUDA with matched diagnostics; identical eight-panel reduction. [final layout: possibly 4 selected panels each]

The interpolation orders differ (quadratic is OSIRIS's production configuration; ArcWarden is linear+filter), but Section 5.3 shows by direct measurement that interpolation order moves the OSIRIS-CUDA rate by only 7%, so the comparison is dominated by architecture, not arithmetic. [optional: quadratic shapes in the tiled framework (est. 20–30% cost) for exact shape parity]

5.2 Ablation: where the speedup comes from

Table 3 is a single-binary, single-session measurement: all five configurations ran back-to-back from one executable (checksum recorded), one git commit, one driver, in one overnight session — full-length runs (206,897 steps each) with matched diagnostics, every configuration a one-line change to the same input deck (the `tile_sort` and `tile_migrate_fused` runtime knobs). `scripts/repro/perf_sweep.sh` regenerates the table and records the provenance (git commit, binary checksum, driver version) in the archived CSV; the run logs, deck snapshots, run metadata, and energy histories for every point are committed with the repository (`docs/perf_sweep_2026-08-12/`).

Table 3: Ablation chain: full-length benchmark runs, one binary, one session, matched diagnostics. Each row is a one-line deck change (`tile_sort`, `tile_migrate_fused`).

configuration	wall time	particle-steps/s	gain
flat: fused kernel, global-atomic Esirkepov	8564.9 s	3.77×10^9	—
+ 16×16 tile sort (every 20) + shared apron deposit, separate migration kernel	2260.8 s	1.43×10^{10}	3.8×
+ migration fused into scatter write-back	1900.8 s	1.70×10^{10}	+19% (4.5× total)

The chain attributes the gain to both design elements: the shared-apron deposit with the amortized sort contributes 3.8×, and fusing the migration pass (periodic wrap + cell recompute) into the scatter write-back contributes a further +19% — eliminating a separate streaming pass over the full particle set, and with it a second read and write of every particle's position. (An early development-time estimate of +6.6% for this element, taken on a different code state, is superseded by this controlled measurement. We note for honesty that an isolated re-measurement of the no-fusion row on a different build in a different session gave 2947 s rather than 2261 s — a 23% spread that is exactly why this table is restricted to one binary and one session, and why the fairness protocol of Section 8 requires repetitions.) The ablation runs are *physically consistent* in the following defined sense (measured values in `energy_consistency.csv`, archived with the sweep): taking the relative difference of the total field energy $|W - W_{\mathrm{ref}}|/W_{\mathrm{ref}}$ between each run and the fused reference, all paths agree to $\lesssim 3 \times 10^{-5}$ through the early linear phase ($t \leq 300\,\omega_{pe}^{-1}$), then diverge chaotically through the exponential-growth phase — float atomic-summation reordering amplified at the instability growth rate — reaching the 10^{-3}–10^{-2} level in and after saturation, while saturated amplitudes and spectra agree at the same few-percent level. Bit-level identity between paths holds only for the first few steps and is not claimed.

Two supporting microbenchmarks: the tiled charge deposit alone is 6.4× faster than global atomics (14.9 vs 2.3 Gdep/s); the plain L2-served push runs at 35.4 Gpush/s while the shared-memory field-staging variant of the same push measures 0.82× — the negative result behind the fields-live-in-L2 rule (Section 2.2). Sort cadence is a flat dial ($N_{\mathrm{sort}} = 10/20/40 \to 1.62/1.70/1.74 \times 10^{10}$ in the same session).

5.3 The classic design is overhead-bound on this hardware

Three independent measurements on the OSIRIS-CUDA reference identify per-step particle book-keeping, not computation, as its pacing item: (i) GPU utilization is 52% during the production run, sampled over minutes; (ii) interpolation order is nearly free — switching linear→quadratic ($\sim 2\times$ deposit arithmetic, $2.25\times$ stencil) moves the rate only from 6.3 to 5.9×10^9 (-7%), which a compute-bound code cannot do; (iii) chunk size is an exact null — doubling `chunk_size` 512→1024 leaves the rate at 5.92×10^9 in both cases, so the cost scales with the pipeline stages themselves, not chunk granularity. The per-step find/exchange/compact pipeline is precisely the term ArcWarden deletes (Section 2.5). During the same benchmark ArcWarden sustains 100% GPU utilization at $\approx 575\,\mathrm{W}$.

5.4 Kernel-level budget

The step decomposes as $T_{\text{step}} = T_{\text{sort}} + T_{\text{deposit + push}} + T_{\text{field}} + T_{\text{diag}}$. On the Yee branch the fused particle kernel is 87.7% of all GPU time (median 9.3 ms/step); the amortized sort contributes 0.64 ms/step; the entire Maxwell update including nine filter passes totals ≈ 0.05 ms/step (0.5%). The GPU spends essentially all of its time on the irreducible particle work. Table 4 gives the equivalent budget for the Darwin branch, whose per-step tile sort is a documented legacy (36% of the step) with a validated upgrade path to the amortized scheme. [ncu report: SOL memory/compute %, occupancy, L2 hit rate of the gathers (profiling/collect_ncu.sh, needs perf-counter permission)]

Table 4: Darwin-branch per-step kernel budget at benchmark scale (Nsight Systems averages over 300 steps).

kernel	ms/step	share
per-step tile sort (legacy; to be amortized)	11.6	36%
fused gather + Boris push	8.0	25%
ρ+J tiled deposit	3.4	11%
$\langle uu \rangle$ (amu) moment deposit	3.3	10%
$\dot{J}$ (dcu) moment deposit	3.0	9%
migration (legacy; fused into scatter in the Yee branch)	2.1	7%
spectral solves (all cuFFT + k-space kernels)	0.17	0.5%

5.5 Model choice as a performance multiplier: the Darwin branch

The Darwin branch runs the benchmark-scale problem at $\Delta t = 0.1\,\omega_{pe}^{-1}$ — $6.9\times$ the Yee light-CFL step — at a measured 5.0×10^9 particle-steps/s, so the full benchmark physical time costs ≈ 943 s: per unit of *physical time simulated*, $2.2\times$ cheaper than the already accelerated Yee branch and $5.7\times$ cheaper than the OSIRIS-CUDA reference, on problems whose physics the model captures. The point is architectural: on a single GPU, choosing the minimal field model for the physics at hand *stacks* with every kernel optimization, and the modular solver layer (Section 2.6) is what makes that choice a one-line deck edit rather than a different code. [scaling curves: rate vs ppc (100–4096) and vs grid (256^2–2048^2), both codes — planned Fig.]

6 Modular physics capabilities

The second claim of the paper is that the performance of Section 5 does not come at the price of rigidity. The 2D capabilities in Table 5 share the same particle engine, deposit framework, and correctness gates (the 1D hybrid rows share the framework's building blocks; Section 3.4); each was added as a module against the tested core, and each is selected at runtime by a deck. A single physics setup can be moved between field models by editing one line (`[field] model = es | darwin | yee`), which is also how the cross-branch validations of Section 4.1 are run.

Table 5: Physics modules and where each is demonstrated.

capability	implementation	demonstrated application
electrostatic	spectral Poisson	two-stream, bump-on-tail (§4)
Darwin EM	spectral, moment-closed (3)	whistler pump / An et al. (§7.1)
full EM	Yee + Esirkepov	anisotropy whistlers + EAW (§5.1)
inhomogeneous B_0	prescribed $B_0(h)$: parabolic, dipole	mirror/dipole chorus (§7.2, §7.3)
anisotropic / loss-cone f_0	configurable loader; (E, μ) mirror map	chorus excitation (§7.2, §7.3)
δf species	weight evolution about analytic f_0	low-noise growth, seeded runs
hybrid kinetic–fluid	linearized cold fluid + hot PIC	1D chorus model (§3.4)
absorbing field boundary	Umeda masked layers [16]	open field lines (§7.2)
particle boundary	reflection / absorption at domain ends	bounce motion, precipitation
wave driver	traveling-wave pump; triggering antenna	An et al.; triggered emissions

Two design choices make the module set cheap to extend. First, the compile-time configuration (`SimConfig`) specializes the kernels — precision, shape, deposit policy, background-field capability — so a module pays only for what it uses; a run without B_0 compiles to the same kernel it had before B_0 existed. Second, the correctness gates of Section 2.8 are cumulative: every module lands with its own gate (e.g. the δf mirror equilibrium hold, the boundary-reflection benchmark with $R < 1\%$ across the whistler band), and all previous gates must keep passing, so the physics surface grows without eroding the tested core.

7 Advanced nonlinear demonstrations

The remaining two levels of the validation ladder move from *correct* to *scientifically capable*: reproduction of an established nonlinear kinetic benchmark, then a demonstration at the frontier of what kinetic codes are asked to do.

7.1 Published nonlinear benchmark: whistler-driven electron trapping (An et al. 2019)

The Darwin branch reproduces the whistler-pump simulations of An et al. [8] — a like-for-like model comparison, since that work used the spectral-Darwin UPIC code, the same field model ArcWarden implements. All three simulations run with exactly the published physical parameters of their Table I — plasma composition, temperatures, ω_{ce}/ω_{pe}, propagation angle, pump mode and frequency, and the target wave amplitude. The one implementation coefficient that is not a physical parameter, the antenna drive strength, is tuned (as in the original work) so that the driven whistler follows the published $\delta B/B_0$ trajectory; the wave amplitude is the physical input, and everything downstream of it is parameter-free. A traveling-wave pump drives a quasi-parallel whistler; when its amplitude crosses $\delta B/B_0 \approx 0.1$, cyclotron-resonant trapping deforms the electron distribution and a secondary electrostatic band erupts — beam-mode Langmuir waves when the resonant velocity sits far out on the tail ($v_r = 3.2\,v_{th}$, Sim 1), electron-acoustic waves and unipolar structures at mid-distribution (Sim 2), and phase-space holes with bipolar fields when it sits in the thermal core ($v_r \approx v_{th}$, Sim 3). ArcWarden reproduces the mechanism and the amplitude threshold in all three cases; Fig. 4 shows Sim 3 at 268M particles: the chain of trapping vortices at $v_r = 1.04\,v_{th}$ with the bipolar $\delta E_\parallel$ signature aligned to the holes, the electrostatic band erupting at 28× the noise floor once the whistler crosses threshold.

This benchmark certifies more than a growth rate: phase-space structure, nonlinear trapping, electron holes, and broadband secondary spectra — the nonlinear vocabulary the framework exists to compute.

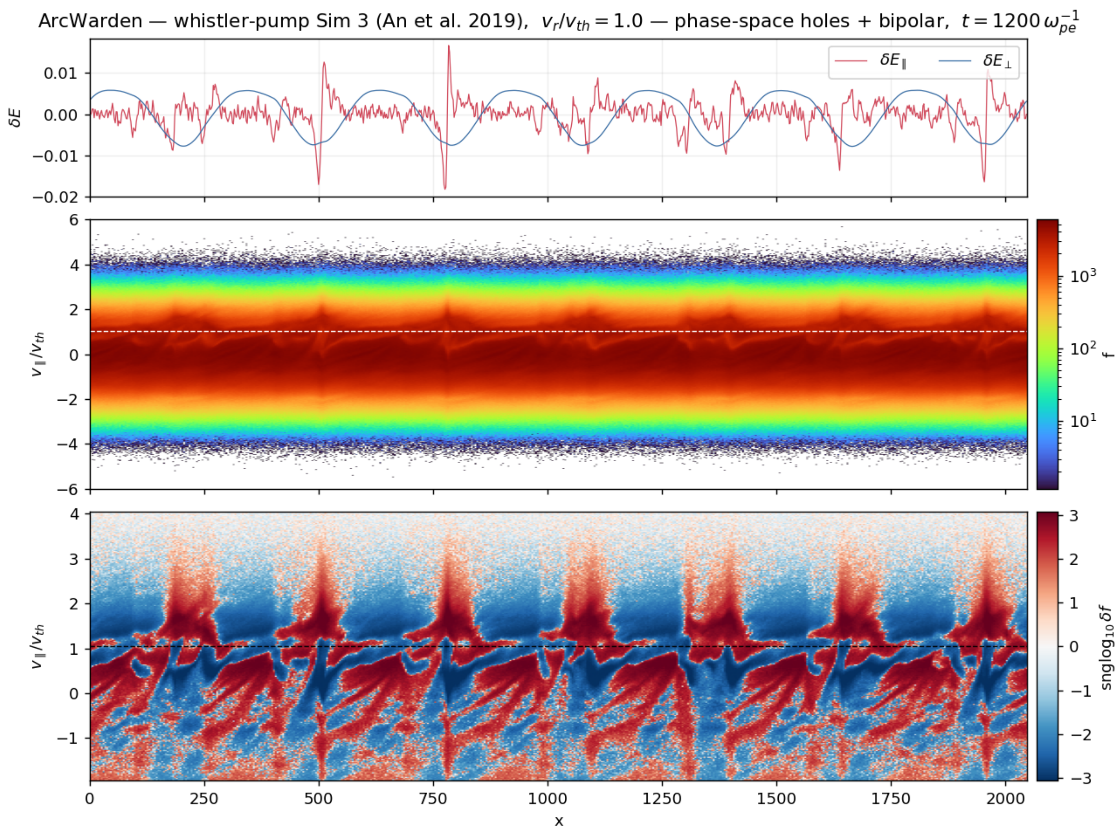

Figure 4: An et al. [8] Sim 3 on the Darwin branch (268M particles): bipolar $\delta E_\parallel(x)$ (top), electron phase space $f(x, v_\parallel)$ (middle), and signed δf (bottom) showing the chain of trapping vortices at $v_r = 1.04\,v_{th}$.

7.2 Capability demonstration: self-consistent coherent whistler waves with frequency chirping in an inhomogeneous plasma

As the top of the ladder we demonstrate a composite capability that exercises most of Table 5 at once: an inhomogeneous (mirror) background field $B_0(h)$, an anisotropic hot electron population held in a mirror equilibrium, absorbing field boundaries and reflecting particle boundaries, and a triggering antenna — the setup of Tao et al. [9] for rising-tone chorus. The 1D electron-hybrid branch self-consistently generates coherent whistler-mode wave elements whose frequency *chirps* upward, together with the resonant phase-space hole whose motion accompanies the sweep (Fig. 5); a δf null test (no anisotropy drive $\rightarrow$ no element) confirms the emission is physical, not numerical. We call this a *triggered capability demonstration*, not a strict reproduction, and Table 6 states the agreement and the deviations side by side. Two setup notes: we adopt a hot-electron density $n_h/n_c = 0.6\%$ — the value stated by Wu et al. [18] from the same group — because with the 6% printed in Ref. [9] the linear growth is so fast that the band saturates long before the published element window (we read the 6% as a misprint); and our triggering antenna is not amplitude-tuned to theirs, which we believe drives the amplitude excess in the table.

Table 6: Triggered chirping demonstration vs Tao et al. [9].

quantity	Tao et al. 2017	ArcWarden
element frequency band	$0.27 \rightarrow 0.67\,\Omega_e$	$0.25 \rightarrow 0.7\,\Omega_e$
chirp rate $\partial\omega/\partial t$	$5.3 \times 10^{-4}\,\Omega_e^2$	$4.2 \times 10^{-4}\,\Omega_e^2$ (-21%)
element window	$t \approx 1133$–$1886\,\Omega_e^{-1}$	$t \approx 600$–$2000\,\Omega_e^{-1}$ (earlier onset)
peak amplitude	—	$\delta B/B_0 \approx 0.05$, $\sim$3–5$\times$ theirs (un-tuned trigger)
subpackets	period 92–130 Ω_e^{-1}	quasi-periodic envelope modulation
phase-space hole	at v_R, tracks $\omega(t)$	tracks $u_R(\omega(t))$ within island width
hot density n_h/n_c	6% (printed) / 0.6% [18]	0.6%

We present this as a *capability demonstration*, not a study of the chirping mechanism: frequency chirping is exquisitely sensitive to errors in resonant particle dynamics [15, 19, 2], which makes it the strongest integration test available for a whistler code — every module in the chain (mirror force, equilibrium load, boundaries, cold-fluid dispersion, hot-particle resonance) must be right simultaneously, or the tone does not form. Physics questions about chorus generation are deliberately out of scope here and are the subject of separate work.

7.3 Spontaneous chorus in a dipole field (Chen et al. 2026)

A second, independent chirping benchmark removes the antenna entirely. Chen et al. [10], using the gcPIC-δf code — a third code lineage, distinct from both UPIC and DAWN — showed that at fixed ω_{pe}/Ω_e three whistler regimes (incoherent band, discrete rising-tone chorus elements, hiss-like emission) are selected by the hot loss-cone population's density and anisotropy alone, with the waves growing spontaneously from noise in a compressed dipole field. ArcWarden reproduces the full regime taxonomy of their Fig. 1 from published parameters (decks `chen2026_case{1,2,3}.ini`: dipole $B_0(\lambda)$, loss-cone mirror equilibrium, absorbing boundaries — every geometry module of Table 5 at once), and the saturation amplitudes reproduce quantitatively where the paper reports them off-equator.

Figure 6 shows the Case II demonstration run at GPU-limit resolution (425M markers, equatorial ppc $= 1.1 \times 10^5$, 3.5 h on the same RTX 5090): the shot-noise floor drops to $\delta B/B_0 \sim 5 \times 10^{-5}$, low enough for a textbook quiet linear phase (measured $\gamma = 2.6 \times 10^{-3}\,\Omega_e$), after which discrete rising-tone elements sweep $0.2 \rightarrow 0.6\,\Omega_e$ with subpacket structure, saturating at $\delta B/B_0 = 0.026$ off-equator ($\lambda = -10^\circ$) against the published 0.027 at $\lambda = 10$–15°. No antenna, no seed field: the elements are generated entirely self-consistently from the loss-cone free energy in the inhomogeneous field. One caveat, stated plainly: onset times are earlier than published and the elements recur, both of which track the initial noise level (a full-f load cannot reach the quoted 10^{-5} seed

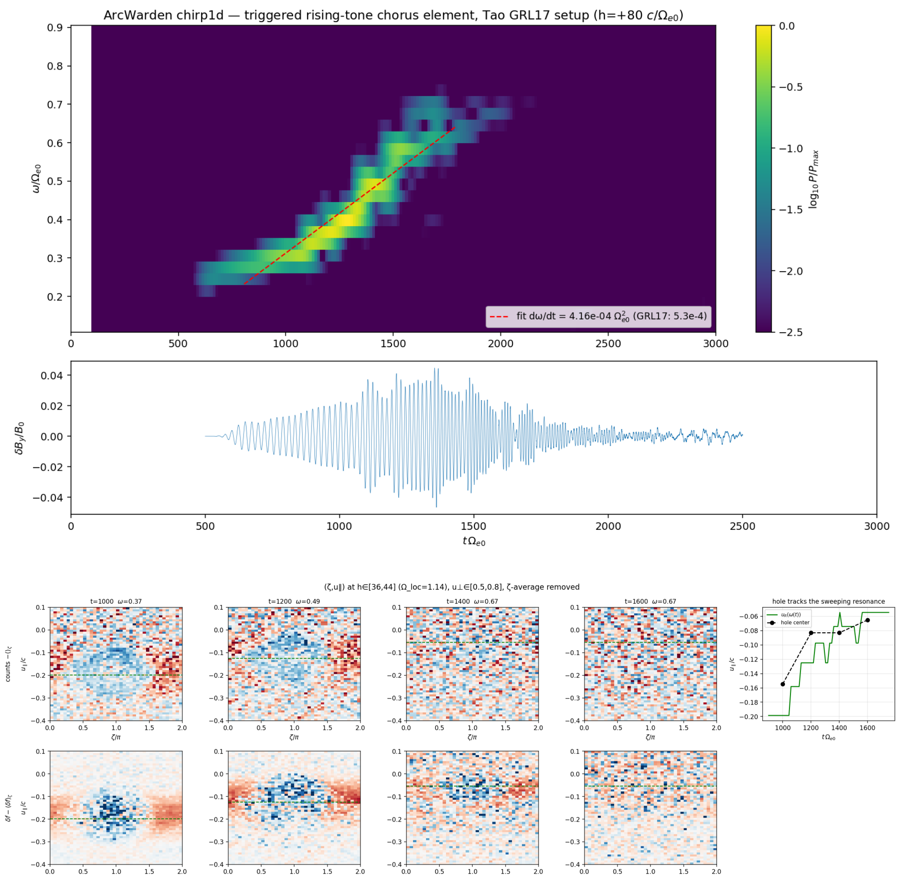

Figure 5: Electron-hybrid branch, setup of Tao et al. [9]. Top: dynamic spectrum $B_w(f,t)$ of the self-consistently generated rising-tone element. Bottom: tracking of the resonant phase-space hole that accompanies the frequency sweep. [final figure selection/cropping; consider adding the $B_0(h)$ profile + loss-cone f_0 as a setup panel]

level of a δf code); the regime taxonomy and saturation amplitudes are insensitive to this. As with Section 7.2, mechanism questions are out of scope here. For the time-to-science frame: the 323-run parameter scan of their Fig. 4 costs an overnight at ArcWarden throughput.

8 Discussion

Design philosophy and division of labor. ArcWarden does not aim to replace massively parallel general-purpose PIC frameworks; OSIRIS, VPIC, PIConGPU, and WarpX serve problems that need thousands of GPUs, and nothing here competes with that. ArcWarden optimizes a different quantity: *time-to-science on one GPU* — the wall-clock and iteration cost of an electron-scale kinetic study that fits a single modern device, including the cost of adding the next physics module. The benchmark result should be read in that frame: a 156M-particle, 207k-step kinetic run finishing in 35 minutes on a $2k consumer device changes what a single researcher can scan in a week.

Scope of the L2 argument. "Fields live in L2" holds for 2D electron-scale grids up to roughly 1600^2–2000^2 on a 96 MB device. For larger grids or 3D, the gather working set exceeds L2 and tile-locality of field reads becomes valuable again — but via the same amortized stale-tolerant sort already present, not via per-step chunk bookkeeping; the stale-tolerant-sort result is the part we expect to transfer unchanged to any architecture, including the exascale codes.

Comparison fairness. OSIRIS ran its production configuration (quadratic shapes); ArcWarden ran linear+filter — the same physical problem and parameters, not identical numerics. The overhead-bound measurements (Section 5.3) bound the effect of this difference at $\sim$7% of the OSIRIS rate, but the definitive comparison is a protocol, which we commit to before submission: identical particle shape and current filter (a quadratic tiled deposit in the `Sink` framework), both codes rebuilt against the same CUDA driver, GPU clocks pinned, and at least three repetitions per configuration with the spread reported. The OSIRIS input file used for the present comparison is included in the repository (`comparison/osiris/`).

Limitations and roadmap. All results are single-GPU, single-precision hot path, 2D3V (1D for the hybrid branch). The Darwin branch still sorts every step (36% of its budget, Table 4) with a validated upgrade path. Planned extensions — multi-GPU domain decomposition and the dipole-field L-shell geometry — reintroduce halo exchange, which the fused kernel tolerates by design (the stray fallback already handles out-of-tile particles).

Development methodology. ArcWarden was developed by one physicist working with an LLM coding agent (Claude Code, Anthropic) under explicit human-written discipline: milestones with quantitative acceptance gates stated before the work, correctness tests preceding any optimization, a committed performance baseline every kernel change must beat, and published figures as acceptance tests. The gates of Section 2.8 and the validation ladder of this paper are that discipline made visible; the full development history is preserved in the repository. We found the human role concentrated into verification — writing gates, judging physics plausibility, catching stability-limit violations — and we note the practice as one reproducible answer to the question of how agent-written GPU code can be trusted in scientific computing.

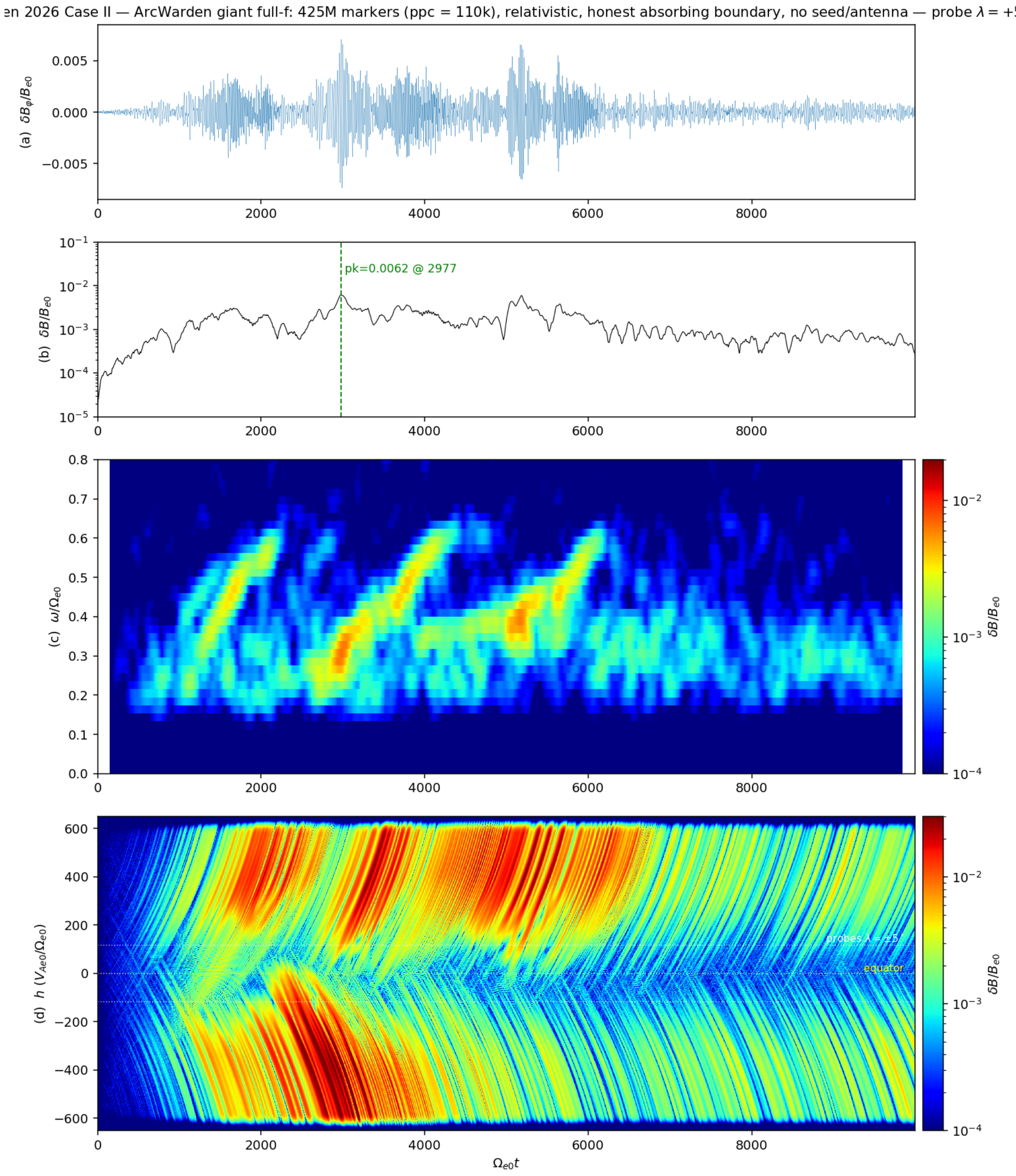

Figure 6: Spontaneous chorus, Case II of Chen et al. [10] at 425M markers: probe waveform (a), wave envelope with the measured linear growth rate (b), dynamic spectrum showing repetitive discrete rising tones $0.2 \to 0.6\,\Omega_e$ with subpackets (c), and $\delta B(h, t)$ along the field line (d). No antenna or seed field is applied.

9 Conclusions

ArcWarden demonstrates that a PIC framework designed around what modern GPUs actually provide — cache-resident fields, fused step kernels, shared-memory scatter aprons, and amortized stale-tolerant sorting — is simultaneously fast, extensible, and scientifically capable: 2.6× the throughput of an established GPU-PIC framework on the same benchmark problem, with the gain accounted element by element; a runtime-modular physics layer in which field models, background geometries, distributions, and boundaries share one tested particle engine; and a validation ladder that runs from textbook dispersion through published nonlinear benchmarks to self-consistent coherent whistler chirping in an inhomogeneous plasma. Electron-scale kinetic studies that were cluster jobs are now interactive desktop work, and the framework is built so the next physics module lands as a deck and a gate, not a fork.

Acknowledgments

Development used Claude Code (Anthropic) under the methodology of Section 8. [grants, computing, colleagues]

Code and data availability

ArcWarden is available at [repository URL on publication]. All input decks (grouped per paper section in `decks/`), analysis scripts, profiling reports, and the OSIRIS input files used for the comparison are included in the repository; every figure in this paper maps to a named deck (see `decks/README.md`).

References

[1] C. K. Birdsall and A. B. Langdon. *Plasma Physics via Computer Simulation.* CRC Press, 2004.

[2] Y. Omura. Nonlinear wave growth theory of whistler-mode chorus and hiss emissions in the magnetosphere. *Earth, Planets and Space*, 73:95, 2021.

[3] R. A. Fonseca, L. O. Silva, F. S. Tsung, V. K. Decyk, W. Lu, C. Ren, W. B. Mori, S. Deng, S. Lee, T. Katsouleas, and J. C. Adam. OSIRIS: a three-dimensional, fully relativistic particle in cell code for modeling plasma based accelerators. In *Computational Science — ICCS 2002*, volume 2331 of *Lecture Notes in Computer Science*, pages 342–351. Springer, 2002.

[4] K. J. Bowers, B. J. Albright, L. Yin, B. Bergen, and T. J. T. Kwan. Ultrahigh performance three-dimensional electromagnetic relativistic kinetic plasma simulation. *Physics of Plasmas*, 15(5):055703, 2008.

[5] M. Bussmann, H. Burau, T. E. Cowan, A. Debus, A. Huebl, G. Juckeland, T. Kluge, W. E. Nagel, R. Pausch, F. Schmitt, U. Schramm, J. Schuchart, and R. Widera. Radiative signatures of the relativistic Kelvin–Helmholtz instability. *Proceedings of SC13: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 5:1–5:12, 2013. (PIConGPU).

[6] L. Fedeli, A. Huebl, F. Boillod-Cerneux, T. Clark, K. Gott, C. Hillairet, S. Jaure, A. Leblanc, R. Lehe, A. Myers, C. Piechurski, M. Sato, N. Zaïm, W. Zhang, J.-L. Vay, and H. Vincenti. Pushing the frontier in the design of laser-based electron accelerators with groundbreaking mesh-refined particle-in-cell simulations on exascale-class supercomputers. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2022. (WarpX; ACM Gordon Bell Prize).

[7] V. K. Decyk and T. V. Singh. Particle-in-cell algorithms for emerging computer architectures. *Computer Physics Communications*, 185(3):708–719, 2014.

[8] X. An, J. Bortnik, B. Van Compernolle, V. Decyk, and R. Thorne. Unified view of nonlinear wave structures associated with whistler-mode chorus. *Physical Review Letters*, 122:045101, 2019.

[9] X. Tao, F. Zonca, and L. Chen. Identify the nonlinear wave-particle interaction regime in rising tone chorus generation. *Geophysical Research Letters*, 44:3441–3446, 2017.

[10] Xu Chen, Huayue Chen, Yu Lin, X. Zhao, X. Shi, and X. Wang. Computational study of whistler-mode wave excitation in the Earth's inner magnetosphere. *Physics of Plasmas*, 33:072105, 2026. TODO verify author list and initials against the publisher.

[11] J. P. Boris. Relativistic plasma simulation — optimization of a hybrid code. In *Proceedings of the Fourth Conference on Numerical Simulation of Plasmas*, pages 3–67. Naval Research Laboratory, Washington, DC, 1970.

[12] C. W. Nielson and H. R. Lewis. Particle-code models in the nonradiative limit. *Methods in Computational Physics*, 16:367–388, 1976.

[13] K. S. Yee. Numerical solution of initial boundary value problems involving Maxwell's equations in isotropic media. *IEEE Transactions on Antennas and Propagation*, 14(3):302–307, 1966.

[14] T. Zh. Esirkepov. Exact charge conservation scheme for particle-in-cell simulation with an arbitrary form-factor. *Computer Physics Communications*, 135(2):144–153, 2001.

[15] Y. Katoh and Y. Omura. Computer simulation of chorus wave generation in the Earth's inner magnetosphere. *Geophysical Research Letters*, 34:L03102, 2007.

[16] T. Umeda, Y. Omura, and H. Matsumoto. An improved masking method for absorbing boundaries in electromagnetic particle simulations. *Computer Physics Communications*, 137:286–299, 2001.

[17] D. Ma, X. An, X. Tao, J. Bortnik, X.-J. Zhang, and V. Angelopoulos. Nonlinear interactions between electron acoustic waves and electron cyclotron harmonic waves / whistler waves. *Physics of Plasmas*, 31:022304, 2024. arXiv:2308.03938; TODO verify final title.

[18] Y. Wu, X. Tao, F. Zonca, L. Chen, and S. Wang. Effect of plasma density on the generation of chorus waves in the Earth's magnetosphere. *Geophysical Research Letters*, 47:e2020GL087791, 2020. TODO verify title/authors/volume against the publisher.

[19] M. Hikishima, S. Yagitani, Y. Omura, and I. Nagano. Full particle simulation of whistler-mode rising chorus emissions in the magnetosphere. *Journal of Geophysical Research*, 114:A01203, 2009.